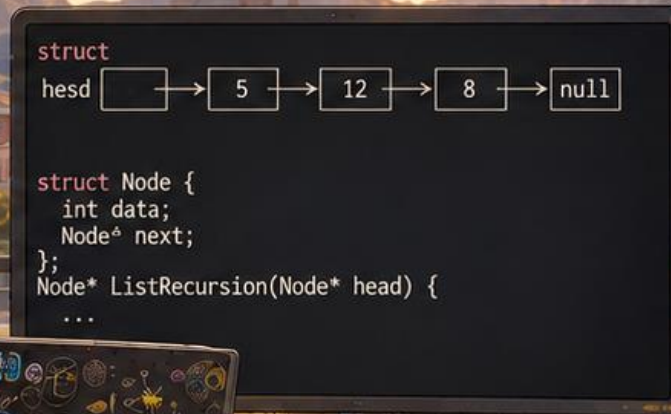




CC1002 - Introducción a la Programación

CLASE 9 : ESTRUCTURAS

Departamento de Ciencias de la Computación (DCC) - FCFM
UNIVERSIDAD DE CHILE



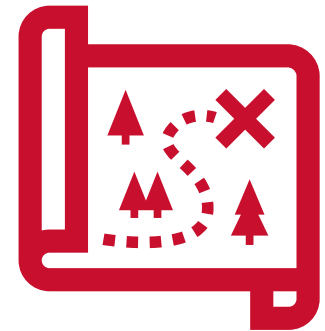
Listas Recursivas

Eduardo Godoy (egodoy@dcc.uchile.cl)

Nadia Decar (nadiadecar@ug.uchile.cl)

Búsqueda del tesoro

- El pirata Barba Azul escondió su gran tesoro. Desconfiado, ideó un plan para proteger el lugar del tesoro.
 - Para encontrarlo había que seguir un largo y secreto camino.
- Al morir, entregó una carta a su hijo. La carta señalaba una dirección, indicando que allí estaba el tesoro. Su ambicioso hijo partió inmediatamente a buscar el tesoro.
- Al llegar a la dirección encontró otra carta, la que contenía una nueva dirección.
- Al llegar a la nueva dirección encontró otra carta, la que contenía una nueva dirección.
- ... esto se repitió ... y se repitió ...
- hasta que finalmente llegó a la dirección donde estaba el tesoro.



Búsqueda del tesoro

- Barba Azul creó una **lista recursiva**
- En cada dirección hay
 - Una carta con nueva dirección, o bien,
 - No hay carta (fin de la lista) → llegó al tesoro 😊
- (carta (carta (carta (. . . ())))) = n elementos
- total de elementos: $1 + \underbrace{\text{largo}(\text{resto})}_{n-1}$



Búsqueda del tesoro

- Lcartas: (carta (carta (carta (. . . ()))))
- Notar que:
 - Sólo se puede sacar la **primera** carta
 - Al sacar una carta, se accede al camino del **resto** de las cartas
- Sea "Lcartas" la lista de cartas
 - carta = **cabeza**(Lcartas) # recupera la primera carta
 - resto = **cola**(Lcartas) # recupera el resto de L,
sin la primera carta
 - **listaVacía** == Lcartas # es verdadero si L está vacía
 - • **esListaVacía**(Lcartas) # es verdadero si L está vacía

Búsqueda del tesoro

- Usando las funciones de lista, determine cuántas direcciones visitó el hijo de Barba Azul para llegar al tesoro

```
def largo(Lcartas) :  
    ::
```

Búsqueda del tesoro

- Usando las funciones de lista, determine cuántas direcciones visitó el hijo de Barba Azul para llegar al tesoro

```
def largo(Lcartas) :  
    ::
```

```
# digitos: int -> int  
# ::  
def digitos(numero) :  
    if numero // 10 == 0 :  
        return 1  
    else :  
        return 1 + digitos( numero // 10)
```

Búsqueda del tesoro

- Usando las funciones de lista, determine cuántas direcciones visitó el hijo de Barba Azul para llegar al tesoro

```
def largo(Lcartas) :  
    if listaVacía == Lcartas : # no hay cartas  
        return 0  
    else:
```

Búsqueda del tesoro

- Usando las funciones de lista, determine cuántas direcciones visitó el hijo de Barba Azul para llegar al tesoro

```
def largo(Lcartas) :  
    if listaVacia == Lcartas : # no hay cartas  
        return 0  
    else:  
        resto = cola(Lcartas)  
        return 1 + largo( resto) # 1 + el largo del resto
```

Búsqueda del tesoro

- Usando las funciones de lista, determine cuántas direcciones visitó el hijo de Barba Azul para llegar al tesoro

```
def largo(Lcartas) :  
    if listaVacia == Lcartas : # no hay cartas  
        return 0  
    else:  
        resto = cola(Lcartas)  
        return 1 + largo( resto) # 1 + el largo del resto
```

```
# digitos: int -> int  
# ::  
def digitos(numero) :  
    if numero // 10 == 0 :  
        return 1  
    else :  
        return 1 + digitos( numero // 10)
```

Listas recursivas: definición

- Módulo listas.py
 - Disponible en uCursos

```
from lista import *
```

```
# Diseño de la estructura
```

```
# lista : valor (cualquier tipo) siguiente (lista)
```

```
estructura.crear("lista", "valor siguiente")
```

estructura.**crear**("lista", "valor siguiente")

Listas recursivas: creación

ListaVacía

- Lista sin elementos

$L_0 = \text{listaVacía}$

estructura.**crear**("lista", "valor siguiente")

Listas recursivas: creación

- Lista sin elementos
 $L0 = \text{listaVacia}$
- Lista con 1 elemento
 $L1 = \text{crearLista}(\text{carta1}, L0)$

listaVacia

(carta1 listaVacia)



estructura.**crear**("lista", "valor siguiente")

Listas recursivas: creación

- Lista sin elementos

`L0 = listaVacía`

- Lista con 1 elemento

`L1 = crearLista(carta1, L0)`

- Lista con 2 elementos (tres opciones):

`L2 = crearLista(carta2, L1)`

`L2 = crearLista(carta2, crearLista(carta1, L0))`

`L2 = crearLista(carta2, crearLista(carta1, listaVacía))`

listaVacía

(carta1 listaVacía)

(carta2 (carta1 listaVacía))

estructura.**crear**("lista", "valor siguiente")

Listas recursivas: creación

- Lista sin elementos

L0 = listaVacía

- Lista con 1 elemento

L1 = crearLista(carta1, L0)

- Lista con 2 elementos (tres opciones):

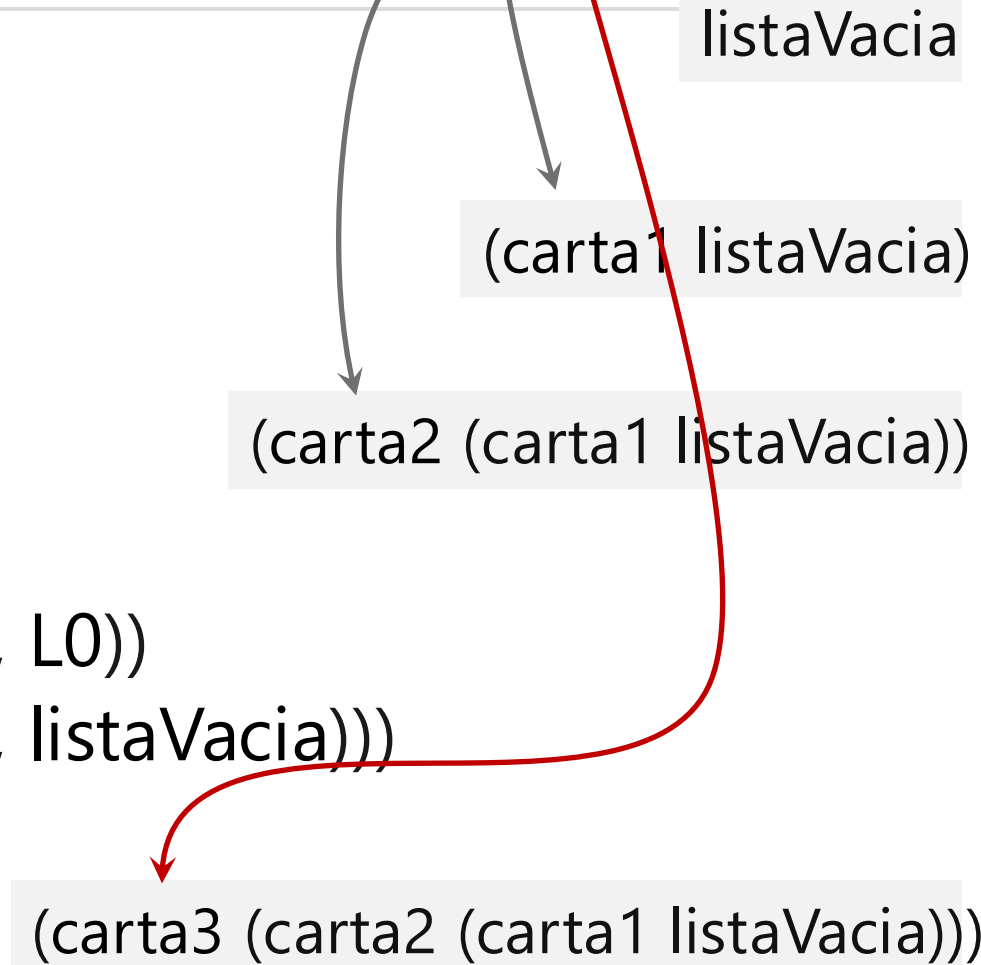
L2 = crearLista(carta2, L1)

L2 = crearLista(carta2, crearLista(carta1, L0))

L2 = crearLista(carta2, crearLista(carta1, listaVacía)))

- Lista con 3 elementos

Lcartas = crearLista(carta3, L2)



Listas recursivas: creación

- Lista con 3 elementos

Lcartas = **crearLista**(carta3, L2)

- Primer valor

cabeza(Lcartas)

→ carta3

cabeza
(carta3 (carta2 (carta1 listaVacia)))
cola

Listas recursivas: creación

- Lista con 3 elementos

Lcartas = **crearLista**(carta3, L2)

- Primer valor

cabeza(Lcartas)

→ carta3

- El resto de los valores

cola(Lcartas)

→ (carta2 (carta1, listaVacia))

cabeza
(carta3 (carta2 (carta1 listaVacia)))
cola

Listas recursivas: Valores compuestos

- Pero **¿qué es una carta?**
 - Es un dato compuesto
 - Contiene: calle, ciudad, país.

```
import estructura  
estructura.crear("carta", "calle ciudad pais")
```

Observación:

- **carta** : una estructura
- **Lcartas**: Lista que contiene estructuras de tipo carta

Listas recursivas: Valores compuestos

```
estructura.crear("carta", "calle ciudad pais")
```

```
c1 = carta("Lira 123", "Santiago", "Chile")
```

```
c2 = carta("Abaroa 2128", "Calama", "Chile")
```

```
c3 = carta("Vespucio 443", "Arica", "Chile")
```

```
Lcartas = crearLista(c3, crearLista(c2, crearLista(c1, listaVacía))))
```

```
unaCarta = cabeza(Lcartas)    # Se recupera c3
```

```
print ( unaCarta.calle)
```

```
→ Vespucio 443
```

Ahora de verdad 🤪

```
import estructura
from lista import *
estructura.crear("carta", "calle ciudad pais")
```

```
c1 = carta("Lira 123", "Santiago", "Chile")
c2 = carta("Abaroa 2128", "Calama", "Chile")
c3 = carta("Vespucio 443", "Arica", "Chile")
```

```
L0 = listaVacia
L1 = crearLista(c1, L0)
L2 = crearLista(c2, L1)
Lcartas = crearLista(c3, L2)
```

```
print ("Carta 1= ", c1)
print ("Carta 2= ", c2)
print ("Carta 3= ", c3)
print
print ("Lista de Cartas= ", Lcartas)
```

lista : valor (cualquier tipo) siguiente (lista)
estructura.**crear**("lista", "valor siguiente")

Ahora de verdad 🤪

```
import estructura
from lista import *
estructura.crear("carta", "calle ciudad pais")
```

```
c1 = carta("Lira 123", "Santiago", "Chile")
c2 = carta("Abaroa 2128", "Calama", "Chile")
c3 = carta("Vespucio 443", "Arica", "Chile")
```

```
L0 = listaVacia
L1 = crearLista(c1, L0)
L2 = crearLista(c2, L1)
Lcartas = crearLista(c3, L2)
```

```
print ("Carta 1= ", c1)
print ("Carta 2= ", c2)
print ("Carta 3= ", c3)
print _____
print ("Lista de Cartas= ", Lcartas)
```

```
Carta 1= carta(calle='Lira 123', ciudad='Santiago', pais='Chile')
Carta 2= carta(calle='Abaroa 2128', ciudad='Calama', pais='Chile')
Carta 3= carta(calle='Vespucio 443', ciudad='Arica', pais='Chile')
```

```
Lista de Cartas= lista(valor=carta(calle='Vespucio 443',
ciudad='Arica', pais='Chile'),
siguiente=lista(valor=carta(calle='Abaroa 2128', ciudad='Calama',
pais='Chile'), siguiente=lista(valor=carta(calle='Lira 123',
ciudad='Santiago', pais='Chile'), siguiente=None)))
```

```
# lista : valor (cualquier tipo) siguiente (lista)
estructura.crear("lista", "valor siguiente")
```

Listas recursivas: Valores compuestos

- Preguntas a resolver:
 1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?
 2. ¿Cuántas cartas tienen como destino la ciudad de Calama?
 3. Cree una nueva lista de cartas, sin la ciudad de Calama
 4. ¿Cuántas ciudades distintas hay en la lista de cartas?
 5. ¿Cuál es la ciudad más repetida en la lista de cartas?

1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?

- Estrategia:
 - Si la lista esta vacía
 - Se retorna falso
 - En caso contrario:
 - Se revisa la ciudad de la primera carta
 - Si es la ciudad, se retorna True
 - En caso contrario, se revisa el resto de la lista de cartas

1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?

```
def estaLaCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return False
```

1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?

```
def estaLaCiudad(Lcartas, ciudad):
```

```
    if esListaVacia(Lcartas):
```

```
        return False
```

```
    else:
```

```
        # será la ciudad que se busca?
```

```
        unaCarta = cabeza(Lcartas)
```

1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?

```
def estaLaCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return False  
    else:  
        # será la ciudad que se busca?  
        unaCarta = cabeza(Lcartas)  
        if unaCarta.ciudad == ciudad:  
            return True  
        else:
```

1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?

```
def estaLaCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return False  
    else:  
        # será la ciudad que se busca?  
        unaCarta = cabeza(Lcartas)  
        if unaCarta.ciudad == ciudad:  
            return True  
        else:  
            # si no es el caso, se revisa el resto de la lista  
            resto = cola(Lcartas)  
            return estaLaCiudad(resto, ciudad)
```



1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?

```
def estaLaCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return False  
    else:  
        # será la ciudad que se busca?  
        if cabeza(Lcartas).ciudad == ciudad:  
            return True  
        else:  
            # si no es el caso, se revisa el resto de la lista  
            return estaLaCiudad(cola(Lcartas), ciudad)
```



Listas recursivas: Valores compuestos

- Preguntas a resolver:
 1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?
 2. ¿Cuántas cartas tienen como destino la ciudad de Calama?
 3. Cree una nueva lista de cartas, sin la ciudad de Calama
 4. ¿Cuántas ciudades distintas hay en la lista de cartas?
 5. ¿Cuál es la ciudad más repetida en la lista de cartas?

2. ¿Cuántas cartas tienen como destino la ciudad de Calama?

- Estrategia:
 - Si la lista esta vacía, se retorna cero
 - En caso contrario:
 - Se revisa la ciudad de la primera carta
 - Si es la ciudad:
 - Se suma y
 - Se revisa el resto de la lista de cartas
 - Sino:
 - Se revisa el resto de la lista de cartas

2. ¿Cuántas cartas tienen como destino la ciudad de Calama?

```
def contarCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return 0
```

2. ¿Cuántas cartas tienen como destino la ciudad de Calama?

```
def contarCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return 0  
    else:  
        unaCarta = cabeza(Lcartas)  
        resto = cola(Lcartas)
```

2. ¿Cuántas cartas tienen como destino la ciudad de Calama?

```
def contarCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return 0  
    else:  
        unaCarta = cabeza(Lcartas)  
        resto = cola(Lcartas)  
        if unaCarta.ciudad == ciudad:  
            # si es, se incrementa y se cuenta el resto  
  
        else:  
            # si no es, solo se cuenta el resto
```



2. ¿Cuántas cartas tienen como destino la ciudad de Calama?

```
def contarCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return 0  
    else:  
        unaCarta = cabeza(Lcartas)  
        resto = cola(Lcartas)  
        if unaCarta.ciudad == ciudad:  
            # si es, se incrementa y se cuenta el resto  
            return 1 + contarCiudad (resto, ciudad)  
        else:  
            # si no es, solo se cuenta el resto  
            return contarCiudad(resto, ciudad)
```



2. ¿Cuántas cartas tienen como destino la ciudad de Calama?

```
def contarCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return 0  
    else:  
        if cabeza(Lcartas).ciudad == ciudad:  
            # si es, se incrementa y se cuenta el resto  
            return 1 + contarCiudad(cola(Lcartas), ciudad)  
        else:  
            # si no es, solo se cuenta el resto  
            return contarCiudad(cola(Lcartas), ciudad)
```



Listas recursivas: Valores compuestos

- Preguntas a resolver:
 1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?
 2. ¿Cuántas cartas tienen como destino la ciudad de Calama?
 3. Cree una nueva lista de cartas, sin la ciudad de Calama
 4. ¿Cuántas ciudades distintas hay en la lista de cartas?
 5. ¿Cuál es la ciudad más repetida en la lista de cartas?

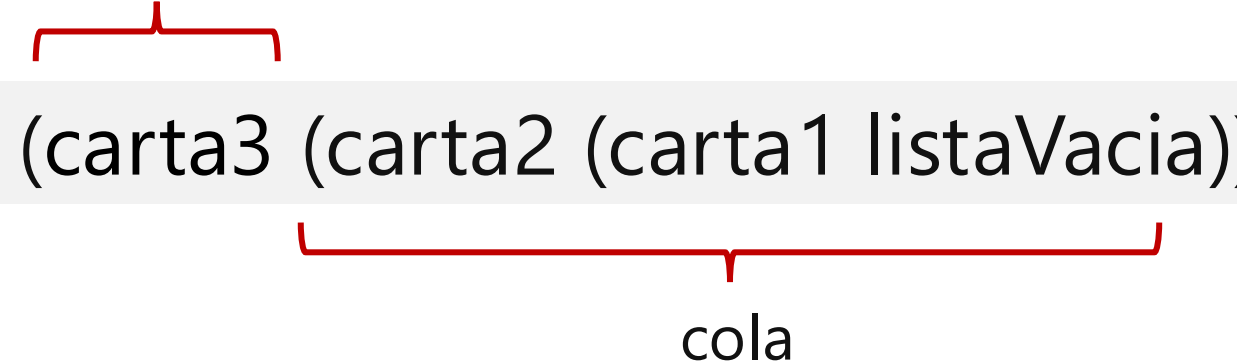
3. Cree una nueva lista de cartas sin la ciudad de Calama

- Estrategia:
 - Si la lista está vacía, se retorna una lista vacía
 - En caso contrario
 - Se recupera la ciudad de la primera carta
 - Si la ciudad **no** es "Calama"
 - Se crea una nueva lista con:
 - 1) La ciudad y
 - 2) La revisión de resto de la lista
 - Se retorna esta nueva lista
 - En caso contrario:
 - Se retorna la lista que se crea al revisar el resto de la lista

3. Cree una nueva lista de cartas sin la ciudad de Calama

```
def sacarCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return listaVacia
```

cabeza



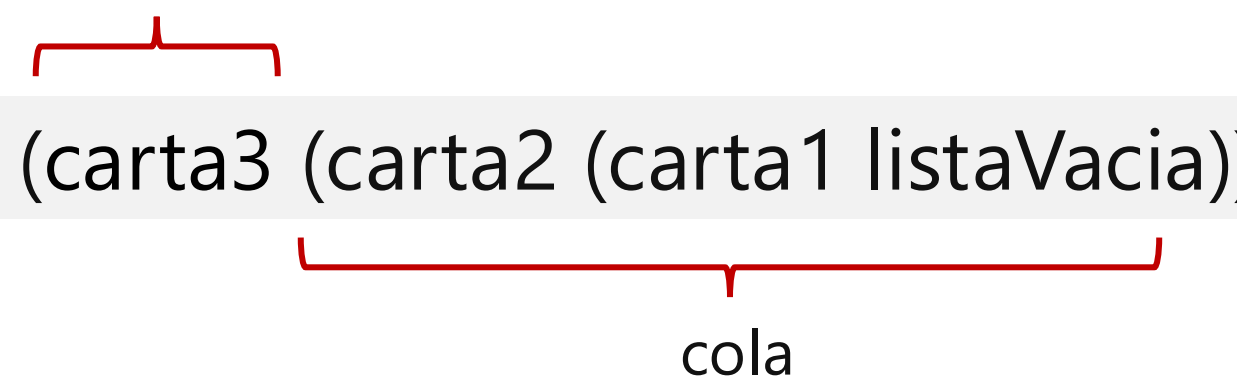
(carta3 (carta2 (carta1 listaVacia)))

cola

3. Cree una nueva lista de cartas sin la ciudad de Calama

```
def sacarCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return listaVacia  
    else:  
        unaCarta = cabeza(Lcartas)  
        resto = cola(Lcartas)
```

cabeza



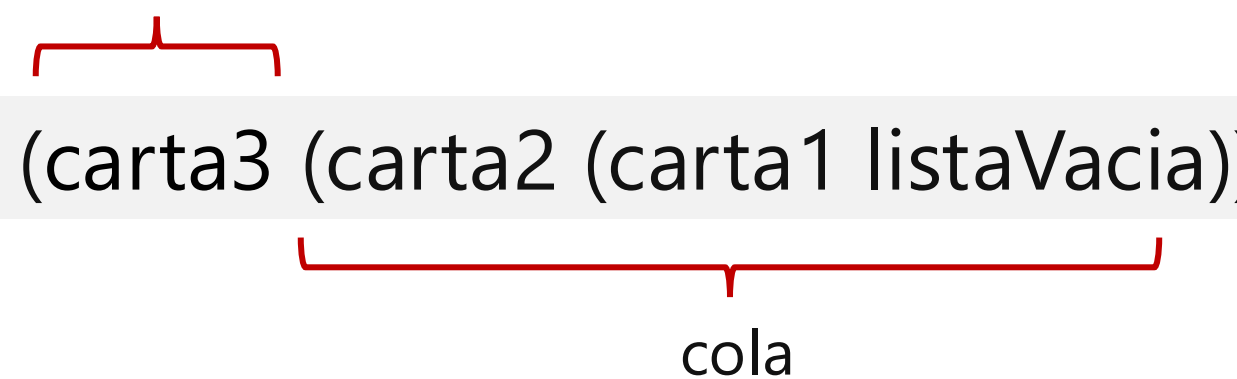
(carta3 (carta2 (carta1 listaVacia)))

cola

3. Cree una nueva lista de cartas sin la ciudad de Calama

```
def sacarCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return listaVacia  
    else:  
        unaCarta = cabeza(Lcartas)  
        resto = cola(Lcartas)  
        if unaCarta.ciudad == ciudad:  
            pendiente....
```

cabeza



(carta3 (carta2 (carta1 listaVacia)))

cola

3. Cree una nueva lista de cartas sin la ciudad de Calama

```
def sacarCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return listaVacia  
    else:  
        unaCarta = cabeza(Lcartas)  
        resto = cola(Lcartas)  
        if unaCarta.ciudad == ciudad:  
            pendiente...  
        else:  
            restoLimpio = sacarCiudad(resto, ciudad)  
            return crearLista(unaCarta, restoLimpio)
```

cabeza

(carta3 (carta2 (carta1 listaVacia))

cola

The diagram shows a list structure. A red bracket labeled 'cabeza' points to the first element of a list. The list is represented as a nested expression: (carta3 (carta2 (carta1 listaVacia))). A second red bracket labeled 'cola' points to the entire nested expression, indicating the rest of the list.

3. Cree una nueva lista de cartas sin la ciudad de Calama

```
def sacarCiudad(Lcartas, ciudad):  
    if esListaVacia(Lcartas):  
        return listaVacia  
    else:  
        unaCarta = cabeza(Lcartas)  
        resto = cola(Lcartas)  
        if unaCarta.ciudad == ciudad:  
            return sacarCiudad (resto, ciudad)  
        else:  
            restoLimpio = sacarCiudad(resto, ciudad)  
            return crearLista(unaCarta, restoLimpio)
```

cabeza

(carta3 (carta2 (carta1 listaVacia))

cola

The diagram illustrates the recursive process of removing an element from a list. A red bracket labeled 'cabeza' points to the first element 'carta3'. Another red bracket labeled 'cola' points to the rest of the list '(carta2 (carta1 listaVacia))'. The entire list structure is highlighted with a light gray background.

Listas recursivas: Valores compuestos

- Preguntas a resolver:
 1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?
 2. ¿Cuántas cartas tienen como destino la ciudad de Calama?
 3. Cree una nueva lista de cartas, sin la ciudad de Calama
 4. ¿Cuántas ciudades distintas hay en la lista de cartas?
 5. ¿Cuál es la ciudad más repetida en la lista de cartas?

4. ¿Cuántas ciudades distintas hay en la lista de cartas?

- Estrategia:
 - Contar sólo la última ciudad no repetida ... Cómo?
- Si la lista esta vacía, se retorna cero
- En caso contrario:
 - Se revisa si la primera ciudad está en el resto de la lista
 - Se usará **estaLaCiudad(. . .)**, ver problema 1
 - Si está, se revisa el resto de la lista
 - Si NO está, se cuenta 1 y se revisa el resto de la lista

4. ¿Cuántas ciudades distintas hay en la lista de cartas?

```
def contarCiudadesDistintas(Lcartas):  
    if esListaVacia(Lcartas):  
        return 0
```

4. ¿Cuántas ciudades distintas hay en la lista de cartas?

```
def contarCiudadesDistintas(Lcartas):  
    if esListaVacia(Lcartas):  
        return 0  
    else:  
        ciudad= cabeza(Lcartas).ciudad  
        resto = cola(Lcartas)
```

4. ¿Cuántas ciudades distintas hay en la lista de cartas?

```
def contarCiudadesDistintas(Lcartas):  
    if esListaVacia(Lcartas):  
        return 0  
    else:  
        ciudad= cabeza(Lcartas).ciudad  
        resto = cola(Lcartas)  
        # si esta repetida, no se contabiliza  
        if estaLaCiudad(resto, ciudad):  
            return contarCiudadesDistintas (resto)
```

4. ¿Cuántas ciudades distintas hay en la lista de cartas?

```
def contarCiudadesDistintas(Lcartas):  
    if esListaVacia(Lcartas):  
        return 0  
    else:  
        ciudad= cabeza(Lcartas).ciudad  
        resto = cola(Lcartas)  
        # si esta repetida, no se contabiliza  
        if estaLaCiudad(resto, ciudad):  
            return contarCiudadesDistintas (resto)  
        else: # dado que no esta, se contabiliza  
            return 1 + contarCiudadesDistintas (resto)
```



Listas recursivas: Valores compuestos

- Preguntas a resolver:
 1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?
 2. ¿Cuántas cartas tienen como destino la ciudad de Calama?
 3. Cree una nueva lista de cartas, sin la ciudad de Calama
 4. ¿Cuántas ciudades distintas hay en la lista de cartas?
 5. ¿Cuál es la ciudad más repetida en la lista de cartas?

5. ¿Cuál es la ciudad más repetida en la lista de cartas?

- Estrategia:
 - Tener un "máximo", con valor inicial cero

5. ¿Cuál es la ciudad más repetida en la lista de cartas?

- Estrategia:
 - Tener un "máximo", con valor inicial cero
 - Tomar la carta de cabeza de la lista y ver cuántas veces (n) está esa ciudad en la lista
 - Se usará **contarCiudad(. . .)**, ver problema 2

5. ¿Cuál es la ciudad más repetida en la lista de cartas?

- Estrategia:
 - Tener un "máximo", con valor inicial cero
 - Tomar la carta de cabeza de la lista y ver cuántas veces (n) está esa ciudad en la lista
 - Se usará **contarCiudad(. . .)**, ver problema 2
 - Si n es mayor, entonces esta ciudad será el nuevo "máximo"
 - Revisar el resto de la lista

5. ¿Cuál es la ciudad más repetida en la lista de cartas?

- Estrategia:
 - Tener un "máximo", con valor inicial cero
 - Tomar la carta de cabeza de la lista y ver cuántas veces (n) está esa ciudad en la lista
 - Se usará **contarCiudad(. . .)**, ver problema 2
 - Si n es mayor, entonces esta ciudad será el nuevo "máximo"
 - Revisar el resto de la lista
- Observación:
 - "máximo" será una nueva **estructura**
 - Contiene la ciudad y las veces que se encontró

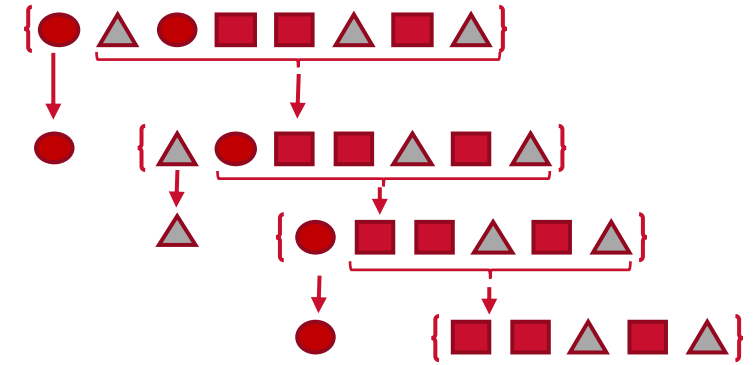
5. ¿Cuál es la ciudad más repetida en la lista de cartas?

```
estructura.crear("maximo", "ciudad total")
```

```
def masRepetida (Lcartas):  
    maxCiudad = maximo("", 0)  
    maxCiudad = buscarMasRepetida(maxCiudad, Lcartas)  
    return maxCiudad.ciudad
```

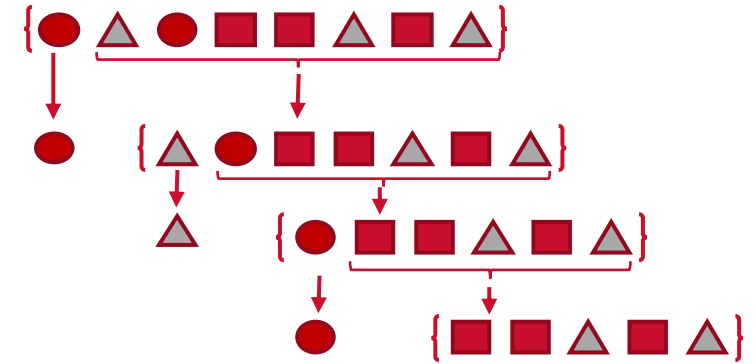
5. ¿Cuál es la ciudad más repetida en la lista de cartas?

```
def buscarMasRepetida (maxCiudad, Lcartas):  
    if esListaVacia(Lcartas):  
        return maxCiudad
```



5. ¿Cuál es la ciudad más repetida en la lista de cartas?

```
def buscarMasRepetida (maxCiudad, Lcartas):  
    if esListaVacia(Lcartas):  
        return maxCiudad  
    else:  
        ciudad = cabeza(Lcartas).ciudad  
        nveces = contarCiudad(Lcartas, ciudad)
```



5. ¿Cuál es la ciudad más repetida en la lista de cartas?

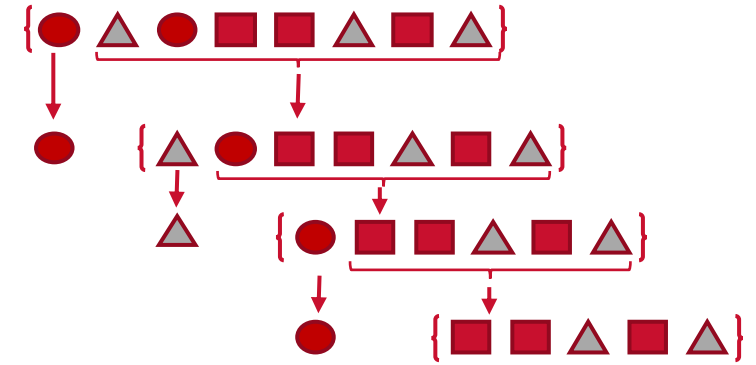
```
def buscarMasRepetida (maxCiudad, Lcartas):  
    if esListaVacia(Lcartas):  
        return maxCiudad  
    else:  
        ciudad = cabeza(Lcartas).ciudad  
        nveces = contarCiudad(Lcartas, ciudad)  
  
        if nveces > maxCiudad.total :  
            maxCiudad= maximo(ciudad, nveces)
```

Recuerde:

estructura.crear("maximo", "ciudad total")

5. ¿Cuál es la ciudad más repetida en la lista de cartas?

```
def buscarMasRepetida (maxCiudad, Lcartas):  
    if esListaVacia(Lcartas):  
        return maxCiudad  
    else:  
        ciudad = cabeza(Lcartas).ciudad  
        nveces = contarCiudad(Lcartas, ciudad)  
  
        if nveces > maxCiudad.total :  
            maxCiudad= maximo(ciudad, nveces)  
  
        return buscarMasRepetida(maxCiudad, cola(Lcartas))
```



Listas recursivas: Valores compuestos

- Preguntas a resolver:
 1. ¿Alguna carta tiene como ciudad, por ejemplo, Calama ?
 - → **Buscar / encontrar un elemento**
 2. ¿Cuántas cartas tienen como destino la ciudad de Calama?
 - → **Contar / Repetición de un elemento**
 3. Cree una nueva lista de cartas, sin la ciudad de Calama
 - → **Duplicar una lista o parte de ella**
 4. ¿Cuántas ciudades distintas hay en la lista de cartas?
 - → **Búsqueda condicionada**
 5. ¿Cuál es la ciudad más repetida en la lista de cartas?
 - → **Búsqueda de máximos / mínimos**
- 

Ejercicio

- Sea L una lista con un número indeterminado de cartas, escriba una función que retorne una lista solo con las cartas que son para el extranjero (no es Chile).
